

C++ 연습 문제: 05. 복사, 이동과 값 의미

1. 기본형이나 단순한 구조체를 복사할 때, 멤버 변수의 값을 그대로 새로운 객체에 전달하는 복사 방식을 무엇이라고 하는가? (하)

1. 얕은 복사(Shallow Copy)
2. 깊은 복사(Deep Copy)
3. 이동 복사(Move Copy)
4. 참조 복사(Reference Copy)

2. 동적 메모리를 소유한 객체를 복사할 때, 단순히 포인터 주소만 복사하면 발생하는 문제로 올바른 것은? (중)

1. 복사 속도가 비정상적으로 느려진다.ㄴ
2. 두 객체가 같은 자원을 가리켜, 소멸 시 이중 해제(Double Free)가 발생할 수 있다.
3. 컴파일러가 자동으로 깊은 복사를 수행하여 메모리 낭비가 발생한다.
4. 포인터의 타입이 암시적으로 변환되어 데이터 손실이 일어난다.

3. 동적 자원을 소유한 객체의 독립성을 유지하기 위해, 새로운 자원을 할당받고 원본의 데이터를 복제하는 복사 방식을 무엇이라고 하는가? (하)

1. 얕은 복사
2. 얕은 참조
3. 깊은 복사
4. 이동 대입

4. 복사 생성자(Copy Constructor)가 호출되는 상황이 아닌 것은? (중)

1. 기존 객체를 사용하여 새로운 객체를 선언하고 초기화할 때
2. 객체를 함수의 값 타입 매개변수로 전달할 때
3. 함수에서 객체를 값 타입으로 반환할 때
4. 이미 생성된 두 객체 사이에서 등호(=)로 값을 대입할 때

5. 복사 생성자를 선언할 때, 매개변수를 상수 참조(const Type&)로 전달해야 하는 가장 큰 이유는 무엇인가? (상)

1. 상수 멤버 함수만 호출할 수 있도록 제한하기 위해서다.
2. 값으로 전달하면 복사 생성자 호출이 무한 루프에 빠지기 때문이다.
3. 객체의 크기를 컴파일 시점에 계산하기 위해서다.
4. 우측값(rvalue)을 일반 참조(&)로 받을 수 없기 때문이다.

6. C++에서 표현식이 식별 가능한 정체성을 가지는지, 이름 없는 임시 값을 나타내는지에 따라 구분되는 개념으로 올바른 것은? (상)

1. public과 private
2. const와 constexpr
3. 좌측값(lvalue)과 우측값(rvalue)
4. static과 extern

7. 자원의 내용을 복제하지 않고, 원본 객체가 소유한 자원의 제어권만 새로운 객체로 넘겨받는 기능을 무엇이라고 하는가? (하)

1. 복사 의미론(Copy Semantics)
2. 이동 의미론(Move Semantics)
3. 참조 계수(Reference Counting)
4. 가상 상속(Virtual Inheritance)

8. 이동 생성자(Move Constructor)의 매개변수 타입으로 사용되는 기호는 무엇인가? (중)

- 1.
2. &
3. &&
4. ->

9. std::move() 함수에 대한 설명으로 올바른 것은? (중)

1. 객체의 실제 데이터를 물리적으로 다른 메모리 위치로 즉시 옮긴다.
2. 전달된 대상을 우측값(rvalue)으로 캐스팅하여 이동 연산이 호출될 수 있게 한다.
3. 객체의 소유권을 영구적으로 삭제하고 메모리를 해제한다.
4. 동적 할당된 배열의 크기를 변경할 때 사용한다.

10. C++의 '3의 규칙(Rule of Three)'에서, 클래스가 동적 자원을 직접 소유할 때 반드시 함께 사용자 정의해야 하는 세 가지 특수 멤버 함수가 아닌 것은? (상)

1. 소멸자(Destructor)
2. 복사 생성자(Copy Constructor)
3. 복사 대입 연산자(Copy Assignment Operator)
4. 기본 생성자(Default Constructor)

11. '5의 규칙(Rule of Five)'은 '3의 규칙'에 C++11부터 새롭게 도입된 두 가지 함수를 포함한다. 추가된 두 가지 함수로 올바른 것은? (상)

1. 가상 생성자와 가상 소멸자
2. 이동 생성자와 이동 대입 연산자
3. 전방 선언과 친구(friend) 함수
4. 비교 연산자와 형 변환 연산자

12. '0의 규칙(Rule of Zero)'에 대한 설명으로 올바른 것은? (중)

1. 클래스는 멤버 변수를 단 하나도 가져서는 안 된다.
2. 생성자, 소멸자 등을 모두 '= delete'로 삭제해야 한다.
3. 동적 자원 관리를 std::string, std::vector, 스마트 포인터 등에 위임하여, 특수 멤버 함수를 직접 작성하지 않는 설계 원칙이다.
4. 포인터를 사용할 때 항상 0(nullptr)으로 초기화해야 한다는 규칙이다.

13. 복사 대입 연산자를 구현할 때, 자기 자신에 대한 대입인지(if (this == &other)) 가장 먼저 검사하는 주된 이유는 무엇인가? (상)

1. 성능을 최적화하기 위해서다.

2. 상수 객체의 값을 변경하지 못하도록 막기 위해서다.
3. 기존 자원을 해제한 뒤 원본 데이터를 복사할 때, 원본이 자신일 경우 데이터가 먼저 날아가는 것을 막기 위해서다.
4. 컴파일러 경고를 없애기 위해서다.

14. 자원의 소유권을 이동시킨 후, 자원을 뺀 원본 객체(Moved-from object)의 상태 처리로 가장 권장되는 방식은 무엇인가? (중)

1. 즉시 소멸자가 호출되도록 메모리를 수동으로 해제한다.
2. 이동 대상과 동일한 데이터를 복제하여 계속 유지한다.
3. 자원을 가리키던 포인터를 `nullptr` 등으로 초기화하여 소멸자가 안전하게 호출될 수 있는 비어 있는 상태로 둔다.
4. 객체의 메모리 주소를 쓰레기값으로 덮어쓴다.

15. 특정 객체가 자원을 독점해야 하거나 복사 비용이 너무 커서, 외부에서 객체를 아예 복사하지 못하게 막고 싶을 때 사용하는 문법은 무엇인가? (하)

1. 복사 생성자 뒤에 `= default;` 를 붙인다.
2. 복사 생성자 뒤에 `= delete;` 를 붙인다.
3. 클래스 이름 앞에 `static`을 붙인다.
4. 복사 생성자를 `public`으로 선언한다.

16. 객체가 마치 내장 타입처럼 복사, 대입, 비교 등이 가능하고, 복사본이 원본과 완전히 독립적으로 동작하도록 설계하는 방식을 무엇이라고 하는가? (중)

1. 참조 의미론(Reference Semantics)
2. 이동 의미론(Move Semantics)
3. 값 의미론(Value Semantics)
4. 가상 의미론(Virtual Semantics)

17. 이동 생성자나 이동 대입 연산자를 구현할 때, 내부에서 예외가 발생하지 않음을 컴파일러에게 알려 성능 최적화를 돕는 키워드는 무엇인가? (중)

1. `noexcept`
2. `constexpr`
3. `explicit`
4. `mutable`

18. 다음 중 우측값(rvalue)으로 취급되는 것은 무엇인가? (상)

1. 동적 할당되어 이름을 가지고 있는 포인터 변수
2. 참조자(&)를 통해 반환된 객체
3. `int x = 10;` 선언 후 사용되는 변수 `x`
4. `GetName()` 함수가 값 타입으로 반환하여 이름이 없는 임시 문자열 객체

19. 클래스가 깊은 복사나 이동을 통해 자원을 직접 관리하는 대신, 자원 소유권을 공유(Shared)하거나 비소유 참조(Weak/Raw Pointer)로 남기는 설계를 선택하는 이유는 무엇인가? (중)

1. 동적 할당된 메모리의 용량을 늘리기 위해서다.
2. 객체의 수명과 접근 방식에 따라 복사보다 참조나 공유가 논리적으로 타당한 경우가 있기 때문이다.
3. C++ 컴파일러는 객체의 깊은 복사를 지원하지 않기 때문이다.
4. 스레드 간의 통신 속도를 무조건 향상시키기 위해서다.

20. 이미 존재하는 객체의 상태를 버리고, 임시 객체나 다른 객체의 자원을 이동 연산으로 덮어쓸 때 호출되는 특수 멤버 함수는 무엇인가? (하)

1. 복사 생성자
2. 복사 대입 연산자
3. 이동 생성자
4. 이동 대입 연산자